

Chapter - 2

Number System.

Introduction

Computers communicate and operate in the binary digits 0 and 1. On the other hand, human beings generally use decimal number system with 10 digits from 0-9. Other number systems are also used such as octal with 8 digits from 0-7, and hexadecimal with digits from 0-15. In the hexadecimal system, digits 10-15 are represented as A-F respectively to avoid confusion with the ~~dec~~ decimal numbers 10-15.

A positional scheme is ^{usually} used to represent a number in any of the number system. This means that each digit will have its value according to its position in number. The number of digit in a position is also ~~ref~~ referred to as base or radix.

for eg:-

binary system has base 2. eg:- $(10101)_2$ ^{Hundreds}

decimal system has base 10. eg:- $(348)_{10}$ ^{Tens} $\Rightarrow 3 \times 100 + 4 \times 10 + 8 \times 1$
_{ones $\Rightarrow 8$}

octal system has base 8 eg:- $(629)_8$

hexadecimal system has base 16 eg:- $(3E9)_{16}$

Different Number Systems

1. Decimal Number System (Base-10 or Radix-10)

It uses 10 ~~gl~~ digits i.e. 0-9 i.e. it uses 10 different symbols to represent the number.

eg:- 6532^0 , 65.23, etc.

$$* \quad 6532^0 = 6 \times 10^3 + 5 \times 10^2 + 3 \times 10^1 + 2 \times 10^0$$

$$* \quad 65.23^0 = 6 \times 10^1 + 5 \times 10^0 + 2 \times 10^{-1} + 3 \times 10^{-2}$$

2. Binary Number System (Base or Radix - 2)

It uses only two different symbols to represent the number i.e. 0 and 1.

eg:-

$$* \quad 11001^0 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$* \quad 1010.011^0 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3}$$

3. ^{Number} Octal ~~Decimal~~ System (Base or Radix - 8)

It uses 8 different symbols i.e. 0, 1, 2, 3, 4, 5, 6, 7.

eg:

$$* \quad 1237^0 = 1 \times 8^3 + 2 \times 8^2 + 3 \times 8^1 + 7 \times 8^0$$

4. Hexadecimal Number System (Base or Radix -16)

In this number system, it uses 16 different symbols.

The first 10 of digits are same as in the decimal number system and the letters A, B, C, D, E, F are used for the digits 10, 11, 12, 13, 14, 15 respectively.

eg:-

$$\begin{aligned} * E253^0 &= E \times 16^3 + 2 \times 16^2 + 5 \times 16^1 + 3 \times 16^0 \\ &= 14 \times 16^3 + 2 \times 16^3 + 5 \times 16^1 + 3 \times 16^0 \end{aligned}$$

Decimal	Binary	Octal	Hexadecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

For Hexadecimal

8421 method (adding the no.s and getting the reqd value)

eg:- BE14 $\rightarrow$ 1011111000000100

$\hookrightarrow B = 8+6+1, E = 8+4+2, 1 = 1, 4 = 4$

421 method $\rightarrow$ For Octal

Binary Coded Decimal (BCD)

Decimal	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Base Conversion Method.

1. Binary to Decimal Conversion

$$(11001)_2$$

$$= 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$= 16 + 8 + 0 + 0 + 1$$

$$= (25)_{10}$$

$$(1010)_2$$

$$= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$

$$= 8 + 0 + 2 + 0$$

$$= (10)_{10}$$

2. Octal to Decimal Conversion

1. $(456)_8 \rightarrow (?)_{10}$

$= 4 \times 8^2 + 5 \times 8^1 + 6 \times 8^0$

$= 4 \times 64 + 5 \times 8 + 6 \times 1$

$= 256 + 40 + 6$

$= (302)_{10}$

3. Hexadecimal to Decimal Conversion

1. $(25BE)_{16} \rightarrow (?)_{10}$

$= 2 \times 16^3 + 5 \times 16^2 + B \times 16^1 + E \times 16^0$

$= 2 \times 16^3 + 5 \times 16^2 + 11 \times 16^1 + 14 \times 16^0$

$= 2 \times 4096 + 5 \times 256 + 11 \times 16 + 14 \times 1$

$= 8192 + 1280 + 176 + 14$

$= (9662)_{10}$

4. Decimal to Binary Conversion

1. $(17)_{10} \rightarrow (?)_2$

	2 17	Rem.
	2 8	1
	2 4	0
	2 2	0
	2 1	0
	0	1



$(17)_{10} = (10001)_2$

2. $(41)_{10} = (?)_2$

		Rem
2	41	
2	20	1
2	10	0
2	5	0
2	2	1
2	1	0
	0	1



$\therefore (41)_{10} = (101001)_2$

3. $(41.625)_{10} = (?)_2$

41.625	Rem.
--------	------

$41 = 101001.$

$0.625 \times 2 = 1.25$	1	↓
$0.25 \times 2 = 0.5$	0	
$0.5 \times 2 = 1$	1	

$\therefore (41.625)_{10} = (101001.101)_2$

5. Decimal to Octal Conversion.

1. $(17)_{10} = (?)_8$

		Rem
8	17	
8	2	1
	0	2



$\therefore (17)_{10} = (21)_8$

$$2. (0.96)_{10} \Rightarrow (?)_8$$

$0.96 \times 8 = 7.68$	7
$0.68 \times 8 = 5.44$	5
$0.44 \times 8 = 3.52$	3
$0.52 \times 8 = 4.16$	4
$0.16 \times 8 = 1.28$	1



$$\therefore (0.96)_{10} = 0.75341_8$$

6. Decimal to Hexadecimal Conversion.

$$1. (65)_{10} \Rightarrow (?)_{16}$$

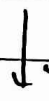
	Rem
$16 \overline{) 65}$	1
$16 \overline{) 4}$	4
0	

$$\therefore (65)_{10} = (41)_{16}$$

$$2. (65.35)_{10} \rightarrow (?)_{16}$$

$$65 = 41$$

$0.35 \times 16 = 5.6$	5
$0.6 \times 16 = 9.6$	9
$0.6 \times 16 = 9.6$	9
$0.6 \times 16 = 9.6$	9



$$\therefore (65.35)_{10} = (41.5999)_{16}$$

7. Hexadecimal to Binary Conversion

1. $(2EA.F37)_{16} = (?)_2$

2	E	A	.	F	3	7
0010	1110	1010		1111	0011	0111.

$$\therefore (2EA.F37)_{16} = (001011101010.111100110111)_2$$

8. Octal to Binary Conversion

1. $(347)_8 = (?)_2$

3	4	7
011	100	111.

$$\therefore (347)_8 = (011100111)_2$$

9. Hexadecimal to Octal Conversion

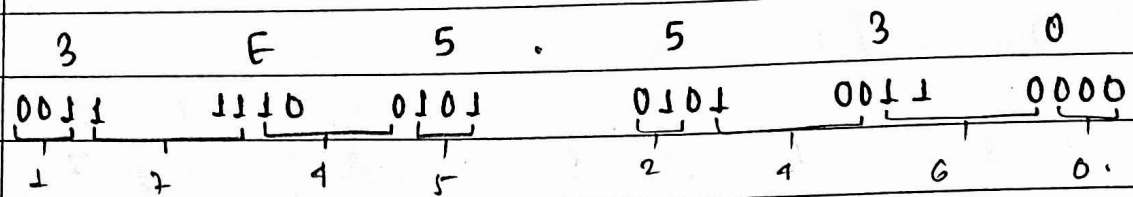
1. $(2EA.F37)_{16} = (?)_8$

2	E	A	.	F	3	7
0010	1110	1010		1111	0011	0111.
1	3	5	2	7	5	6 7

$$(2EA.F37)_{16} = (001011101010.111100110111)_2$$

$$(2EA.F37)_{16} = (1352.7574)_8$$

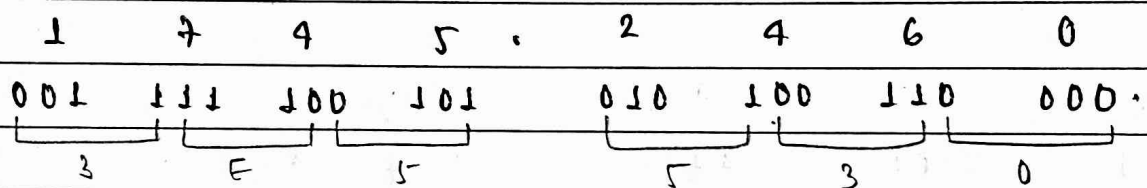
2. $(3E5.530)_{16} = (?)_8$



$(3E5.530)_{16} = (1745.2460)_8$

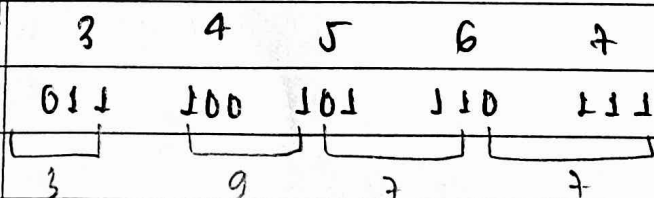
10. Octal to Hexadecimal conversion.

1. $(1745.2460)_8 \rightarrow (?)_{16}$



$\therefore (1745.2460)_8 = (3E5.530)_{16}$

2. $(34567)_8 \rightarrow (?)_{16}$



$\therefore (34567)_8 = (3977)_{16}$

add in this way add in this way

$$\begin{array}{r} 001010 \cdot 110000 \\ \hline \end{array}$$

Date:

Page:

$$(165)_{10} = (?)_{BCD}$$

1	6	5
0001	0110	0101

$$(165)_{10} = (000101100101)_{BCD}$$

$$= (101100101)_{BCD}$$

$$(325.736)_8 = (?)_{10}$$

$$3 \times 8^2 + 2 \times 8^1 + 5 \times 8^0 + 7 \times 8^{-1} + 3 \times 8^{-2} + 6 \times 8^{-3}$$

$$192 + 16 + 5 + 0.875 + 0.046 + \cancel{0.0058} = 0.011$$

$$\cancel{213.926}_{10} = (\cancel{213.926})_{10} = (213.932)_{10}$$

2 this
 213.932

What is the decimal equivalent of octal number $(325.736)_8$?

or

$$(325.736)_8 = (?)_{10}$$

Tuesday
26th April, 2022

Date:
Page:

Binary to Octal

1. $(1010)_2 = (?)_8$

001010
1 2

= $(12)_8$

2. $(10101010.11101)_2 = (?)_8$

010101010.111010
2 5 2 2 2

= $(252.72)_8$

Hex to Binary

1. $(10101010.11101)_2 = (?)_{16}$

10101010.11101000
A A EF 8

= $(AA.E8)_{16}$

2. $(2E4.A6)_{16} = (?)_2$

2 E 4 . A 6
↓ ↓ ↓ ↓ ↓
0010 1110 0100 1010 0110

= $(1011100100.10100110)_2$

$$3. (273.45)_8 = (?)_2$$

$$\begin{array}{ccc} 2 & 7 & 3 \\ \downarrow & \downarrow & \downarrow \\ 010 & 111 & 011 \end{array} \quad \begin{array}{cc} 4 & 5 \\ \downarrow & \downarrow \\ 100 & 101 \end{array}$$

$$\Rightarrow (10111011.100101)_2$$

$$4. (3B3.CD)_{16} = (?)_2$$

$$\begin{array}{ccc} 3 & B & 3 \\ \downarrow & \downarrow & \downarrow \\ 0011 & 1011 & 0011 \end{array} \quad \begin{array}{cc} C & D \\ \downarrow & \downarrow \\ 1100 & 1101 \end{array}$$

$$(\text{1110110011.11001101})_2$$

$$\begin{array}{cccccccccccccccc} 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & . & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & . & 0 \\ \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} \\ 1 & 6 & 6 & 3 & 6 & 3 & 6 & 3 & 6 & 3 & 6 & 3 & & 6 & 3 & 6 & 3 & 2 & 0 & & & & \end{array}$$

$$\Rightarrow (1663.632)_8$$

$$5. (4592)_{10} = (BCD)_2$$

$$\begin{array}{ccc} 4 & 5 & 9 \\ \downarrow & \downarrow & \downarrow \\ 0100 & 0101 & 1001 \end{array} \quad \begin{array}{c} 2 \\ \downarrow \\ 0010 \end{array}$$

$$\Rightarrow (100010110010010)_{BCD}$$

6. $(27.50)_{10} = (?)_2$

$$\underline{27.50}$$

2		27	Rem	
2		13	1	
2		6	1	
2		3	0	↑
2		1	1	
		0	1	

$$0.50 \times 2 = 1.$$

$$\Rightarrow (11011.1)_2$$

7. $(27.50)_{10} = (?)_8$

8		27	Rem	
8		3	3	
		0	3	↑

$$0.50 \times 8 = 4.$$

$$\Rightarrow (33.4)_8$$

8. $(39.3125)_{10} = (?)_{16}$

16		39	Rem	
16		23	7	
		7	2	↑

$$0.3125 \times 16 = 5$$

$$\Rightarrow (27.5)_{16}$$

$$\# (27.58)_{10} = (?)_{16}$$

16	27	Rem
16	1	11
	0	1

$$0.58 \times 16 = 8$$

$$\Rightarrow (1B.8)_{16}$$

Binary Coded Decimal (BCD)

Binary Codes for decimal digits requires a minimum of four bits. It is a straight assignment of binary equivalent to each digit. For eg, 95 when converted into binary is equal to 1011111. But when the same number is represented in BCD code, each decimal digit is represented by 4-bits as 1001 for 9 and 0101 for 5. Thus, the BCD equivalent will be 1001 and 0101.

There are only ten code groups in BCD system i.e.

0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000 and 1001.

eg:-	Decimal	BCD
	0	0000
	1	0001
	9	1001
	10	0001 0000
	11	0001 0001
	99	1001 1001
	100	0001 0000 0000

Merits

1. It is easy to encode and decode decimals into BCD. and vice versa.
2. It is also simple to implement a hardware algorithm for the BCD converter.
3. It is very useful in digital system whenever decimal formation is given either as input or displayed as output.

Demerits

1. BCD requires more bits than straight binary to represent decimal number of more than one digit. This is because BCD doesn't have all possible 4-bit groups and therefore it is inefficient.
2. For the integers greater than nine, BCD occupies more space than the corresponding binary.
eg:-
 $12 \rightarrow 1100$ (Binary)
 $12 \rightarrow 0001 \ 0010$ (BCD)
3. The arithmetic operations using BCD code requires a complex design of ALU than the straight binary system.

ASCII Code.

ASCII is the abbreviation for American Standard Code for Information Interchange. ASCII is a universally accepted alpha numerical code used in the most computers and other electronic equipments. Most computer keyboards are standardized with ASCII codes. When we enter a letter, a number or a control command, the ASCII code goes into the computer. ASCII is a 7-bit code. It can represent 2^7 128 characters.

eg:-

'a' = $97_{10} = 1100001$

'j' = 0110001

'A' = 1000001 = 65_{10} .

Excess-3 Code

It is an important 4-bit code sometimes used with Binary Coded Decimal (BCD) numbers. To convert any decimal numbers into its excess-3 form, we have to add 3 to each decimal digits, and then convert the sum to a BCD number.

eg:-

13

1st: add 3 to each decimal digit

$$\begin{array}{r} 1 \quad 3 \\ + 3 \quad + 3 \\ \hline 4 \quad 6 \end{array}$$

2nd: Convert result into 4-bit BCD number.

4 6
0100 0110 is an equivalent Excess-3 code of number 13.

46

1st: add 3 to each decimal digit.

$$\begin{array}{r} 4 \quad 6 \\ + 3 \quad + 3 \\ \hline 7 \quad 9 \end{array}$$

2nd: convert result into 4-bit BCD number.

7 9
0111 1001 is an equivalent Excess-3 code of no 46,

shaft:-

Date:

Page:

Decimal	BCD	Excess-3
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

Gray Code:

Gray code is unweighted code and it is not an arithmetic code. In Gray code, bits are arranged in such a way that it changes by only one bit as it sequences from one number to next number. A typical application of the Gray code occurs in rotational or shaft encoder, which is used to convert some angular position of a shaft to a digital format.

Binary	Gray Code
000	000
001	001
010	011
011	010
100	110

[# any one bit can be changed randomly.]

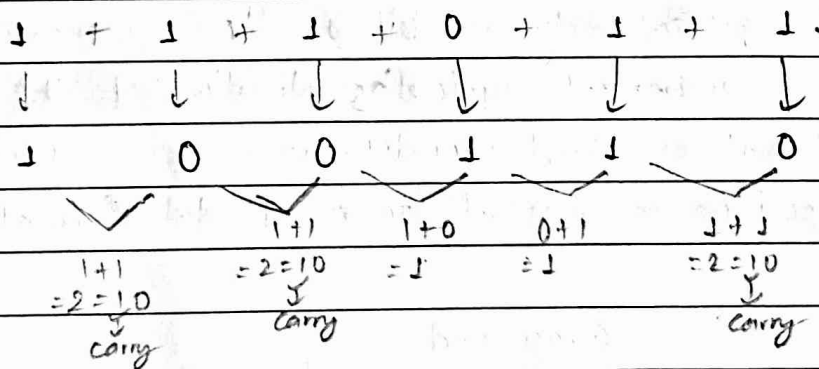
Binary to Gray Conversion.

For conversion of binary number into a gray code number, following rules are applied:

1. The most significant digit (MSB) (or bit) [left most bit] in the gray code is same as the corresponding digit in the binary number.
2. Going from left to right, add each adjacent pair of binary digit to get the next gray code.
3. Discard any carrier.

for example:-

Q. Convert $(111011)_2$ to Gray Code.



$$(111011)_2 = (100110)_{\text{gray}}$$

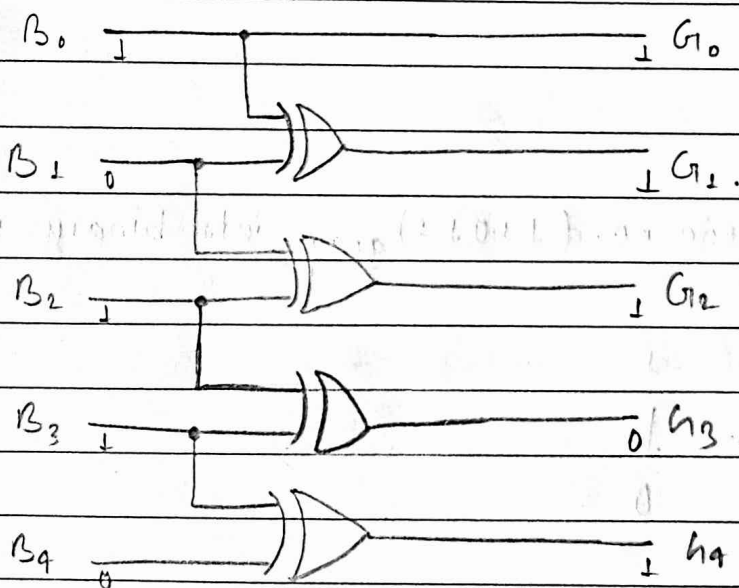
Q. Convert $(10110)_2$ into Gray code.

1 0 1 1 0
↓ ↓ ↓ ↓ ↓

1 1 1 0 1

$(10110)_2 = (11101)_{\text{gray}}$

Binary to Gray Code Converter



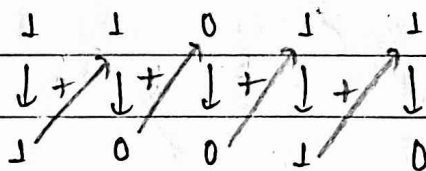
Gray to Binary Conversion

For conversion of gray code number into a binary code, following steps are followed:

1. The Most significant bit (or digit) (MSB) [Left Most Bit] in the gray code is same as the corresponding digit in the binary number.
2. Add each binary digit generated to the gray digit in the next adjacent position.
3. Discard carries.

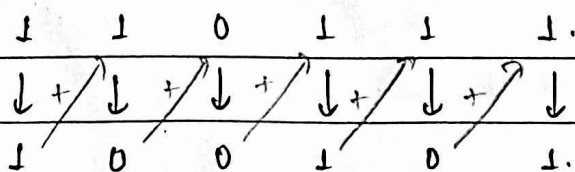
for example:-

Q. Convert the gray code no. $(11011)_\text{gray}$ into binary number.

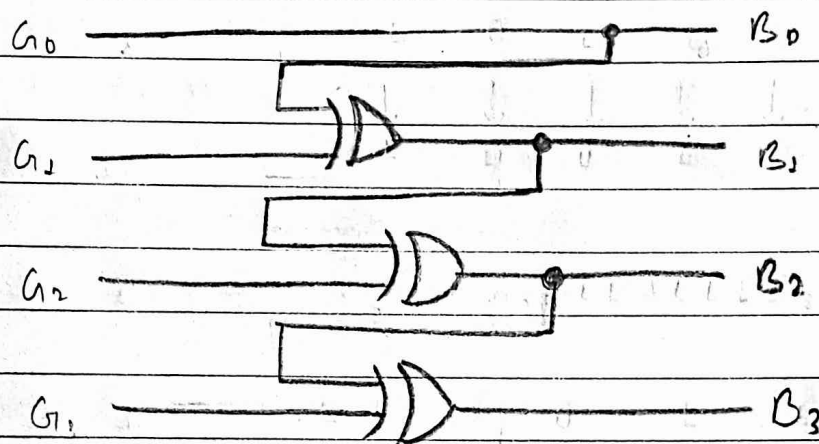


$$(11011)_\text{gray} = (10010)_2.$$

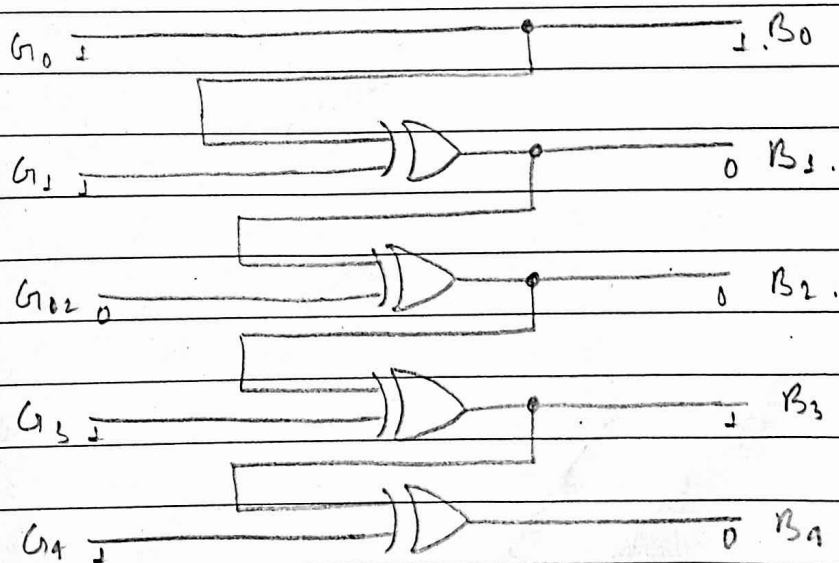
Q. Convert $(110111)_\text{gray}$ to binary.



$$(110111)_\text{gray} = (100101)_2.$$

Gray to Binary Code Converter

(11011) gray to binary code converter.



Q. $(10101101)_2 \rightarrow \text{Gray}$

1	0	1	0	1	1	0	1
↓	↓	↓	↓	↓	↓	↓	↓
1	1	1	1	1	0	1	1

$(10101101)_2 = (11111011)_{\text{gray}}$

1	1	1	1	1	0	1	1
↓	↓	↓	↓	↓	↓	↓	↓
1	0	1	0	1	1	0	1

$(11111011)_{\text{gray}} = (10101101)_2$

Assignment 1

Error Detecting & Correcting Codes.

Buy a thin copy for Assignment copy.

spelling

Date:

Page:

What is weighted & non-weighted code?

weighted code.

The number system which is positionally weighted or the digit position of any number represents a specific weight is known as weighted code.

eg:- decimal, binary, octal, etc.

non-weighted code.

The number system which is not positionally weighted or the digit position of any number does not represent the specific weight is known as non-weighted code.

eg:- gray code, excess-3 code, etc.

~~Signed number~~

~~Signed bit: the left most bit~~

Signed Numbers

A. Signed bit:

The left most bit in the signed binary number is a signed bit which tells us whether a number is positive or negative. A '0' sign bit indicates a positive number and a '1' sign bit indicates a negative number.

eg:-

-70
↓ ↓
1111 0000

↑ sign → -ve

-8
↓
1000 1000

↑ '1' sign → -ve

The negative numbers are represented using 3 different methods:

1. Sign Magnitude Representation
2. 1's Complement Representation.
3. 2's Complement Representation

B. Sign Magnitude:

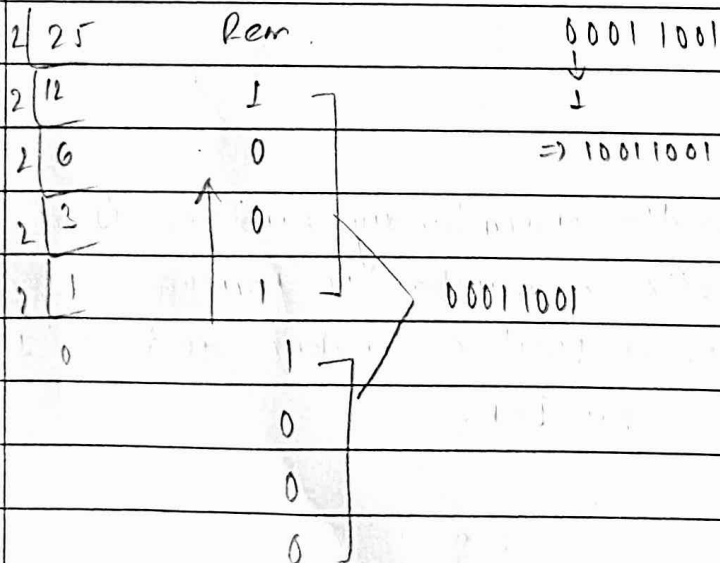
When a signed binary number is represented in a sign magnitude, the left most bit is the sign bit and the remaining bits are magnitude bits.

eg:- The decimal number +25 is expressed as in 8-bits sign binary number using the sign magnitude form as

00011001.

The decimal number -25 is expressed as in 8-bits sign binary number using the sign magnitude form as

10011001.



The only difference between +25 and -25 is the signed bit because the magnitude bits are in true binary form for both positive and negative numbers.

C. 1's Complement Form:

Positive numbers in 1's complement are represented in the same way as the positive sign magnitude numbers. Negative numbers however are the 1's complement of the corresponding positive numbers.

eg:- Using 8-bits, the decimal number -25 is expressed in the 1's complement of +25 (00011001). as
11100110.

D. 2's Complement Form:

Positive numbers in 2's complement form are represented in the same way as the positive sign magnitude numbers. Negative numbers however are the 2's complement of the corresponding positive numbers.

eg:- Using 8-bits, the decimal number -25 is expressed in the 2's complement of +25 (00011001) as 11100111.

Sign Magnitude

Decimal values of a positive and negative numbers in the sign magnitude form are determined by summing the weights in all the magnitude bit positions where there are ones and ignoring the positions where there are zeros.

Q. Determine the decimal value of its signed binary number expressed in sign magnitude.

1. $\overset{\text{sign bit}}{\downarrow} \overset{6}{1} \overset{5}{0} \overset{4}{1} \overset{3}{0} \overset{2}{1} \overset{1}{0} \overset{0}{1}$
 magnitude

7# Floating Point Representation (or Number)

floating point is a numerical representation system in which a string of digits represent a real number. The name floating point refers to the fact that the radix point (decimal point or binary point) can be placed anywhere relative to the digits within the string. A fixed point is of the form $a \times b^n$, where a is the fixed point part referred as mantissa (or significand) of the number. and ' b ' represent the base and ' n ' represents the exponent.

$$\begin{array}{c}
 a \times b^n \rightarrow \text{exponent} \\
 \downarrow \quad \quad \downarrow \\
 \text{mantissa} \quad \text{base} \\
 \text{(or significand)}
 \end{array}$$

* IEEE (Institute of Electronic & Electrical Engineers)

The IEEE standard for floating point.

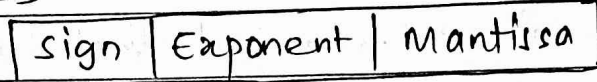
- The IEEE has produced a standard for floating point format arithmetic in mini & micro computers. This standard specifies how single precision (32-bit), double precision (64-bit) and quadruple (128-bit) floating point numbers are to be represented as well as arithmetic should be carried out on them.

General Layout

The three fields in IEEE 754 float :

MSB

LSB



Binary floating point numbers are stored in a sign magnitude form where the MSB bit is the sign bit, exponent is the biased exponent and fraction is the significand. without the Most Significant bit (MSB).

Thursday
May 5th, 2022

Date:
Page:

Binary Arithmetic

1. Binary Addition

Before we can understand how a computer performs arithmetic, we have to learn to add binary numbers because computer circuit process binary numbers. Binary addition is the key to binary subtraction, multiplication & division.

The four basic steps of binary addition are:

$$0 + 0 \rightarrow \text{sum} = 0, \text{ carry} = 0$$

$$0 + 1 \rightarrow \text{sum} = 1, \text{ carry} = 0$$

$$1 + 0 \rightarrow \text{sum} = 1, \text{ carry} = 0$$

$$1 + 1 \rightarrow \text{sum} = 0, \text{ carry} = 1.$$

auxiliary carry $\rightarrow$

$$\begin{array}{r} 1010 \\ + 0011 \\ \hline 1110 \end{array}$$

2. Binary Subtraction

Binary subtraction is the process of subtracting binary numbers 0, 1. The process is same as the arithmetic operation.

$$0 - 0 \rightarrow \text{difference} = 0, \text{ borrow} = 0$$

$$0 - 1 \rightarrow \text{difference} = 1, \text{ borrow} = 1$$

$$1 - 0 \rightarrow \text{difference} = 1, \text{ borrow} = 0$$

$$1 - 1 \rightarrow \text{difference} = 0, \text{ borrow} = 0$$

eg:-

$$\begin{array}{r} 1001 \\ - 0111 \\ \hline 0010 \end{array}$$

3. Binary Multiplication

Binary multiplication is the process of multiplying binary numbers. The process is same as that of arithmetic multiplication of decimal numbers.

$$0 \times 0 = 0$$

$$0 \times 1 = 0$$

$$1 \times 0 = 0$$

$$1 \times 1 = 1$$

4. Binary Division

Binary division is the process of dividing the binary numbers.

$$0 \div 1 = 0$$

$$1 \div 0 = 1$$

5. Complements.

Complements are used in digital computer to simplify the subtraction operation & for the logical manipulation. Simplifying operations lead to similar less expensive circuits to implement the operation.

For base r system, there are two types of complement:

1. The radix complement (r 's complement)

r 's complement of $N = r^n - N$, where $n = \text{no. of digit}$, $N = \text{number}$ and $r = \text{base or radix}$.

2. The diminished radix complement ($r-1$'s complement)

$r-1$'s complement of $N = r^n - r^m - N$, where $N = \text{number}$, $n = \text{no. of digits}$, $m = \text{fractional part of } n \text{ digits}$ and $r = \text{radix}$.

Binary Addition

$$\begin{array}{r} 1 \\ 10010 \\ + 11011 \\ \hline (101101)_2 \end{array}$$

Binary Subtraction

$$\begin{array}{r} 1010 \\ - 1001 \\ \hline (0001)_2 \end{array}$$

Multiplication

$$\begin{array}{r} 110 \\ \times 101 \\ \hline 110 \\ 000 \\ + 110 \\ \hline (11110)_2 \end{array}$$

Division

$$\begin{array}{r} 101 \overline{) 110110} \\ \underline{101} \\ 0011 \\ \underline{0} \\ 111 \\ \underline{101} \\ 0100 \rightarrow \text{Remainder} \end{array}$$

9's Complement.

Q. Subtract 235 from 572 using 9's complement.

$$\begin{array}{r} 999 \\ - 235 \\ \hline \end{array}$$

$$\begin{array}{r} 999 \\ - 235 \\ \hline 764 \end{array} \rightarrow \text{9's complement of 235}$$

$$\begin{array}{r} 764 \\ + 572 \\ \hline \end{array}$$

$$\begin{array}{r} 1336 \\ \hline \end{array}$$

$$\begin{array}{r} 1336 \\ \quad \quad \quad \rightarrow + 1 \\ \hline \end{array}$$

$$\begin{array}{r} 1336 \\ \quad \quad \quad \rightarrow + 1 \\ \hline 1337 \end{array}$$

Q. Subtract 485 from 228 using 9's complement.

$$\begin{array}{r} 999 \\ - 485 \\ \hline \end{array}$$

$$\begin{array}{r} 999 \\ - 485 \\ \hline 514 \end{array} \rightarrow \text{9's complement}$$

$$\begin{array}{r} 514 \\ + 228 \\ \hline \end{array}$$

$$\begin{array}{r} 514 \\ + 228 \\ \hline 742 \text{ (no carry)} \end{array}$$

$$\begin{array}{r} 999 \\ - 742 \\ \hline \end{array}$$

$$\begin{array}{r} 999 \\ - 742 \\ \hline 257 \end{array}$$

10's complement.

Q. Subtraction using 10's complement, subtract 235 from 572.

$$\begin{array}{r}
 999 \\
 - 235 \\
 \hline
 764 \\
 + 1 \\
 \hline
 765 \rightarrow 10's \text{ complement} \\
 + 572 \\
 \hline
 \boxed{1}337 \\
 \swarrow \text{neglect.}
 \end{array}$$

Q. Subtract 485 from 228 using 10's complement.

$$\begin{array}{r}
 999 \\
 - 485 \\
 \hline
 514 \rightarrow 9's \text{ complement} \\
 + 1 \\
 \hline
 515 \rightarrow 10's \text{ complement} \\
 + 228 \\
 \hline
 743 \text{ (no carry)}
 \end{array}$$

$$\begin{array}{r}
 999 \\
 - 743 \\
 \hline
 256
 \end{array}$$

$$\begin{array}{r}
 + 1 \\
 \hline
 - 257
 \end{array}$$

1's complement

Q. Subtract $(1000011)_2$ from $(1010100)_2$ using 1's complement.

Ans:

$$\begin{array}{r} 1010100 \\ - 1000011 \\ \hline 0010001 \end{array}$$

Step 1: 1000011
 $0111100 \rightarrow 1's \text{ complement}$

Step 2: 1010100
 $+ 0111100$
 $\hline 10010000$
 $\swarrow$
 1
 $\hline 10010001$

$\therefore (1010100)_2 - (1000011)_2 = (0010001)_2$

Q. Using 1's complement, subtract $(1000011)_2 - (1010100)_2$.

Step 1: 1010100
 $0101011 \rightarrow 1's \text{ complement}$

Step 2: 1000011
 $+ 0101011$
 $\hline 1101110 \text{ (no carry)}$

Step 3: There is no end carry, so we calculate 1's complement of the ans.
 1101110
 $- 0010001 \rightarrow 1's \text{ complement}$

$$\therefore (1000011)_2 - (1010100)_2 = -(10001)_2$$

Q. Subtract using 1's complement, $(11010)_2 - (110)_2$.

Step 1: 11010 Minuend
 00110 Subtrahend

Step 2: $00110 \rightarrow 11001$

Step 3:

$$\begin{array}{r} 11010 \\ + 11001 \\ \hline 110011 \end{array}$$

Step 4: Since there is overflow so add this to result.

$$\begin{array}{r} 10011 \\ + 1 \\ \hline 10100 \end{array}$$

$\therefore (11010)_2 - (110)_2 = (10100)_2$

2's Complement

Q. Using 2's Complement, subtract $(1000011)_2$ from $(1010100)_2$.

Step 1: 1000011 Minuend
 1000011 Subtrahend.

$\rightarrow 0111100 \rightarrow 1's \text{ complement}$

Step 2:

1010100	0111100
$+ 0111100$	$+ 1$
1001000	0111101

Step 3:

1001000	1010100
$+ 0111101$	$+ 0111101$
1001000	1001001

neglect

$\therefore (1010100)_2 - (1000011)_2 = (100101)_2$

Q. Using 2's complement, subtract $(1000011)_2 - (1010100)_2$.

Step 1: 1010100
 $\rightarrow 0101011 \rightarrow 1's \text{ complement}$

Step 2:

0101011
$+ 1$
0101100

Step 3:

1000011
$+ 0101100$
1101111

Step 4: Since there is no overflow bit, the ans is -ve & 2's complement to the last result.

$1101111 \rightarrow 0010000$

0010000
$+ 1$
0010001

$\therefore (1000011)_2 - (1010100)_2 = -(100101)_2$

Rules

9's complement decimal subtraction

Steps

1. Make both numbers to have same number of bits (Minuend and Subtrahend).
2. Find 9's complement of the number to be subtracted.
3. Add this complement to the first number (minuend).
4. Is there any over-flow bit?
5. If 'Yes', add over flow bit (i.e. end around carry) in last result.
6. If 'No', Answer is negative and 9's complement of the last result.

eg: $(123)_{10}$ from $(345)_{10}$

Step 1: Making equal no. of bits.

345	Minuend
123	Subtrahend.

Step 2: Finding 9's complement of subtrahend.

9 9 9
- 1 2 3

8 7 6 $\rightarrow$ 9's complement.

Step 3: Adding the complement to minuend.

345
+ 876
1221

Step 4: There's overflow bit to add ^{it} to this result.

221
+ 1

 $(222)_{10} //$

10's complement decimal subtraction.

Steps

1.